Appendix for: SchemaMind for Smart Contract Vulnerability Detection

Supplementary Material

This appendix provides supplementary technical artifacts supporting the core mechanisms and experimental rigor of SchemaMind. Section A details the structured representation of knowledge patterns; Section B archives the complete multi-stage prompt repository; Section C expands on the composition of our enhanced benchmark; and Section D presents extended numerical results, including cross-model validation and full ablation data. These resources ensure the reproducibility of our framework in smart contract security auditing.

A Knowledge Pattern Representation

Vulnerable Pattern

```
{
    "vulnerability_name": "Timestamp Dependency",
    "pattern_type": "vulnerable",
    "vulnerability_pattern_description": "The contract uses block.timestamp to determine whether the user is eligible to claim the reward. Since miners can manipulate the timestamps of blocks within a certain range, this may lead to the logic of reward distribution being unfairly advanced or delayed.",
    "trigger_condition": "The judgment conditions of the key business logic of the contract (such as unlocking funds and determining winners) directly or indirectly depend on block.timestamp.",
    "fix_pattern": "For critical logic that requires an exact time sequence, avoid using block.timestamp. One can consider using the block number (block.number) to estimate the time, or introduce an external Oracle service to obtain reliable time."
}
```

Safe Pattern

```
{
    "vulnerability_name": "none",
    "pattern_type": "safe",
    "safety_pattern_description": "This contract effectively follows the 'check - Take effect - interact' pattern by completing all status updates (such as deducting the user balance) before making any external calls. This design fundamentally eliminates the risk of reentry attacks.",
    "mitigation_factor": "Complete the status change before external interaction(State change before external interaction)。",
    "guideline": "Always update all relevant state variables before executing external calls (such as sending Ether) to prevent reentrancy attacks."
}
```

Fig. 1. Example of a Vulnerable Pattern and a Safe Pattern stored in the SRSP-KB.

Figure 1 illustrates the detailed internal structure of a *Vulnerable Pattern* and a *Safe Pattern*. These structured JSON objects are the core components of our SRSP-KB. As described in Section 3.1.2, this structured information is generated during the offline knowledge distillation phase. The `vulnerability_pattern_description` or `safety_pattern_description` field is

crucial, as it captures the abstract logical essence of the code, which is then vectorized to enable our hybrid retrieval mechanism. This textual information, along with vector embeddings of both the original code snippet and the key pattern description field, is what is ultimately stored in each entry of the SRSP-KB.

B Comprehensive Prompt Repository

This section presents the specific prompts used for the different stages of our framework. The following figures showcase the implementation details of the PHCM and the final structured prompt architecture for in-context learning and inference.

- **Figure 2:** The initial, unbiased prompt used to get the LLM's first judgment on a code snippet.
- **Figure 3:** The prompts for the two-stage correction loop in the PHCM. The *Soft Rethink* provides a gentle hint for re-evaluation, while the *Hard Rethink* gives a more direct hint if the former fails. This process is crucial for ensuring the high fidelity of our knowledge base.
- **Figure 4:** The prompt used in the final knowledge distillation step, which instructs the LLM to generate the structured *Vulnerable Pattern* or *Safe Pattern* based on a high-confidence label.
- **Figure 5:** The final prompt architecture used for the inference stage, which combines a base instruction, the Top-K examples retrieved by HSSR, and the target code to guide the LLM's reasoning process.

C Dataset Composition and Complexity

As detailed in Section 4.1.2, we enhanced the evaluation test set with high-complexity benign samples to address the simplicity of benign code in existing benchmarks. This ensures a more realistic and challenging assessment of a model's ability to distinguish between vulnerable and securely-written complex code. Figure 6 provides a representative example of these generated samples. This specific sample correctly implements the Checks-Effects-Interactions pattern to prevent re-entrancy attacks, serving as a non-trivial negative case for the detection models.

Table 1 provides the distribution of the enhanced benchmark. By integrating 1,619 expert-audited complex benign samples, we significantly mitigate the risk of over-optimistic results found in simpler datasets. These samples are designed to mimic vulnerable structures while strictly adhering to safety patterns like Checks-Effects-Interactions.

Table 1. Categories of the test dataset

Category	**None**	**Overflow-Underflow**	**Timestamp-Dependency**	**Re-Entrancy**
Count	1619	1333	1381	1343

D Extended Experimental Results and Ablations

This section provides complete numerical data to support the findings discussed in Section 4.2. To ensure the reproducibility of our experiments, we specify the exact versions of the LLMs used in SchemaMind: • **Qwen-Turbo:** We utilized the specific snapshot version qwen-turbo-2025-07-15 accessed via the Alibaba Cloud DashScope API. • **DeepSeek-V3.1:** The experiments were conducted using the deepseek-v3.1 (released in late 2025) via the DeepSeek official API.

D.1 Ablation Analysis

Table 2 quantifies the contribution of PHCM. The sharp decline in F1-score (5.49 percentage points) without PHCM confirms that offline label correction is indispensable. PHCM ensures that the reasoning stored in the KB is logically consistent with facts, rather than being a stochastic byproduct of LLM's initial judgment.

Table 2. Ablation study on PHCM (K=4, $\alpha : \beta = 7 : 3$, LLM: Qwen-Turbo-2025-07-15)

KB variant	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)
Full SchemaMind	85.52	87.85	86.64	87.24
w/o PHCM	80.63 (-4.89)	84.78 (-3.07)	81.87 (-4.77)	83.30 (-3.94)

D.2 Cross-Model Validation and Tuning

Table 3 demonstrates that SchemaMind consistently yields gains across different LLM lineages (Qwen and DeepSeek), affirming the universality of SRSP-KB. Furthermore, Table 4 reveals that while semantic logic (3:7 ratio) is instructive at lower depths ($K = 2$), structural anchoring (7:3 ratio) becomes paramount as context length increases, preventing inference saturation from implementation variances.

Table 3. Effectiveness of SchemaMind with $\alpha : \beta = 1 : 1$

Model	K	Acc. (%)	Pre. (%)	Rec. (%)	F1 (%)
Qwen-Turbo	0	81.98	83.25	83.12	83.18
	2	84.09	85.19	84.65	84.92
	4	**84.46**	**86.40**	**84.82**	**85.60**
Deepseek-V3.1	0	82.82	87.75	84.00	85.83
	2	84.92	**88.51**	**85.59**	87.03
	4	**85.06**	87.45	85.33	**86.38**

Table 4. Effect of $\alpha : \beta$ ratios and Number of Examples on model performances (LLM: Qwen-Turbo)

K	$\alpha : \beta$	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)
0	NA	81.98	83.25	83.12	83.18
2	0:1	85.71	**88.35**	86.35	87.34
	3:7	84.06	87.99	**86.80**	**87.39**
	1:1	84.09	85.19	84.65	84.92
	7:3	**85.97**	86.75	85.94	86.34
	1:0	85.58	86.83	85.90	86.36
4	0:1	81.02	83.43	81.42	82.41
	3:7	81.02	83.77	81.51	82.62
	1:1	84.46	86.40	84.82	85.60
	7:3	**85.52**	**87.85**	**86.64**	**87.24**
	1:0	82.53	84.52	82.53	83.51
8	0:1	81.47	83.36	81.72	82.53
	3:7	80.01	82.39	80.43	81.40
	1:1	80.33	82.80	80.60	81.68
	7:3	**82.38**	**84.34**	**82.72**	**83.52**
	1:0	78.61	82.34	78.37	80.31
16	0:1	**79.96**	82.49	**80.31**	**81.38**
	3:7	76.97	80.16	77.40	78.76
	1:1	79.86	**82.70**	80.08	81.37
	7:3	77.53	80.94	77.73	79.31
	1:0	77.08	80.53	76.34	78.38

```
You are a smart contract vulnerability detector.
For the following code, please determine if it contains EACH of the listed vulnerabilities: "Reentrancy",
"Timestamp Dependency", "Integer Overflow/Underflow".(it can be more than one)
Your response MUST be a JSON object where each key is a vulnerability type and its value is either "yes" or "no".
Example format:
{
    {"Reentrancy": "no",
     "Timestamp Dependency": "yes",
     "Integer Overflow/Underflow": "no"
    }
}

--- CODE TO ANALYZE ---
{code_snippet}
----------------------

Your JSON response:
```

Fig. 2. Prompt for the Initial Judgment step.

Soft rethink (FN):

Your previous analysis may have been incorrect, potentially missing a vulnerability. Let's think step by step.
Hint: {specific_hint_str}
IMPORTANT: Your response MUST strictly follow the original JSON format with three keys: "Reentrancy", "Timestamp Dependency", and "Integer Overflow/Underflow", and their values must be "yes" or "no". Do NOT add any other keys or nesting.

--- CODE TO ANALYZE ---
{code_snippet}

Your new JSON response:

Soft rethink (FP):

Your previous analysis may have been incorrect, potentially identifying a vulnerability where none exists. Let's think step by step.
Hint: {specific_hint_str}
IMPORTANT: Your response MUST strictly follow the original JSON format with three keys: "Reentrancy", "Timestamp Dependency", and "Integer Overflow/Underflow", and their values must be "yes" or "no". Do NOT add any other keys or nesting.

--- CODE TO ANALYZE ---
{code_snippet}

Your new JSON response:

Hard rethink (FN):

Your previous analysis was incorrect. You failed to identify the vulnerability/vulnerabilities in the code.
The provided code snippet may only have vulnerabilities related to: **{possible_labels_str}**.
Hint: {specific_hint_str}
IMPORTANT: Your response MUST strictly follow the original JSON format with three keys: "Reentrancy", "Timestamp Dependency", and "Integer Overflow/Underflow", and their values must be "yes" or "no". Do NOT add any other keys or nesting.

--- CODE TO ANALYZE ---
{code_snippet}

Your new JSON response:

Hard rethink (FP):

Your previous analysis was incorrect. You incorrectly identified a vulnerability in this code.
The code may be actually SAFE from the vulnerability you identified.
Hint: {specific_hint_str}
IMPORTANT: Your response MUST strictly follow the original JSON format with three keys: "Reentrancy", "Timestamp Dependency", and "Integer Overflow/Underflow", and their values must be "yes" or "no". Confirm why it is safe.

--- CODE TO ANALYZE ---
{code_snippet}

Your new JSON response:

Fig. 3. Prompts for the Progressive Correction Loop (Soft and Hard Rethink).

Vulnerable pattern distillation:

```
You are a world-class smart contract security expert. The following Solidity code snippet IS VULNERABLE to
**{vulnerability_name}**.
Your task is to distill the core FLAW into a structured "Vulnerability Pattern".
IMPORTANT: Your "vulnerability_pattern_description" MUST be concise, under 80 words.
Please respond ONLY with a JSON object in the format:
{
    {"vulnerability_name": "{vulnerability_name}",
     "pattern_type": "vulnerable",
     "vulnerability_pattern_description": "...",
     "trigger_condition": "...",
     "fix_pattern": "..."}
}

--- VULNERABLE CODE TO ANALYZE ({vulnerability_name}) ---
{code_snippet}
```

Safe pattern distillation:

```
You are a world-class smart contract security expert. The following Solidity code snippet IS SAFE from Reentrancy,
Timestamp Dependency, and Integer Overflow/Underflow.
Your task is to distill the core SAFE PRACTICE into a structured "Safety Pattern".
IMPORTANT: Your "safety_pattern_description" MUST be concise, under 80 words.
Please respond ONLY with a JSON object in the format:
{
    {"vulnerability_name": "none",
     "pattern_type": "safe",
     "safety_pattern_description": "...",
     "mitigation_factor": "...",
     "guideline": "..."}
}

--- SAFE CODE TO ANALYZE ---
{code_snippet}
```

Fig. 4. Prompt for the Final Knowledge Distillation step.

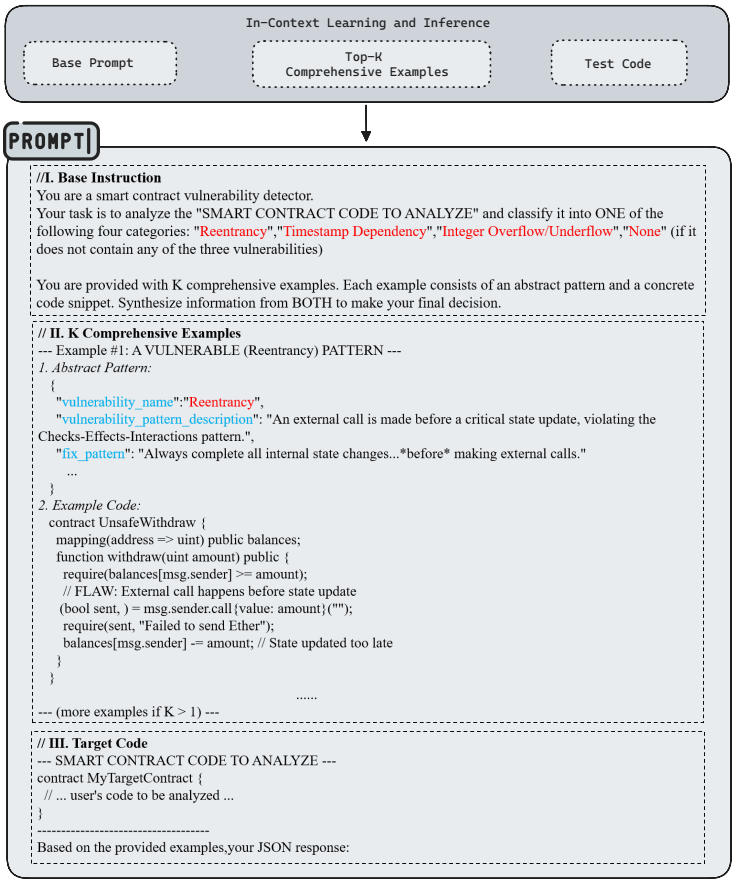

Fig. 5. The structured prompt architecture for the final inference stage.

```
\begin{lstlisting}[language=Solidity, caption={Benign Sample 1: A secure vault correctly implementing the
Checks-Effects-Interactions pattern.}, label={lst:cei-example}]
pragma solidity ^0.8.20;
contract SecureVault {
    mapping(address => uint256) public balances;

    event Deposit(address indexed user, uint256 amount);
    event Withdrawal(address indexed user, uint256 amount);

    function deposit() public payable {
        require(msg.value > 0, "Deposit amount must be greater than zero");
        balances[msg.sender] += msg.value;
        emit Deposit(msg.sender, msg.value);
    }

    function withdraw(uint256 amount) public {
        require(balances[msg.sender] >= amount, "Insufficient balance");
        balances[msg.sender] -= amount;
        emit Withdrawal(msg.sender, amount);

        (bool success, ) = msg.sender.call{value: amount}("");
        require(success, "Failed to send Ether");
    }

    function getContractBalance() public view returns (uint256) {
        return address(this).balance;
    }
}
\end{lstlisting}
```

Fig. 6. Example of a high-complexity benign code snippet.